Smart Parking: Parking Space Detection Using Computer Vision

Anusha S. L. Bandaru, Rakesh R. Bhatija and Tejas Desai

Applied Artificial Intelligence, University of San Diego

Authors Note

This project was undertaken as part of our exploration into the introduction to Artificial Intelligence (AI). As a group, working on real-world datasets and implementing various models for parking space detection has been a rewarding and educational experience.

Throughout the project, We gained hands-on experience with end to end data science lifecycle. Despite the initial learning curve, the process helped us build a stronger foundation in machine learning(ML) and understand the challenges involved in deploying intelligent vision-based systems.

We'd like to thank the open-source community and platforms like Roboflow and PKLot for providing accessible datasets and tools that enable aspiring learners to develop real-world AI solutions.

Abstract

Accurate and efficient detection of empty parking spaces is essential for advancing smart city infrastructure. This study investigates the application of deep learning and computer vision (CV) techniques to automate parking occupancy detection using the PKLot-v2 640 dataset, which contains over 12,000 labeled parking lot images. We implemented and evaluated four deep learning models—a custom convolutional neural network (CNN), ResNet50 using TensorFlow (ResNet50-TF), ResNet50 using PyTorch (ResNet50-Torch), and a Vision Transformer (ViT)—to classify parking spots as either occupied or vacant.

All models were trained and validated using standard evaluation metrics: precision, recall, F1 score, AUC-ROC and accuracy. Optimal classification thresholds were determined for each model. The ResNet50-Torch and ViT models achieved perfect F1 scores (1.00) with AUC-ROC scores of 0.9999 and 0.9998, respectively. The CNN and ResNet50-TF models also demonstrated competitive performance, with F1 scores of 0.93 and 0.96. These findings underscore the potential of transformer-based architectures and fine-tuned residual networks in visual classification tasks.

The study concludes that the ViT model offers the most promise for real-time deployment in intelligent parking systems. The comparative insights can inform model selection for broader smart transportation applications.

Keywords: smart parking, computer vision, Image classification, object detection, image segmentation, deep learning, parking space detection, Vision Transformer, ResNet50, convolutional neural network, precision-recall curve,threshold optimization,AUC-ROC curve,

Methodology:

Methodology of the project and structure of this paper are illustrated in Figure 1.

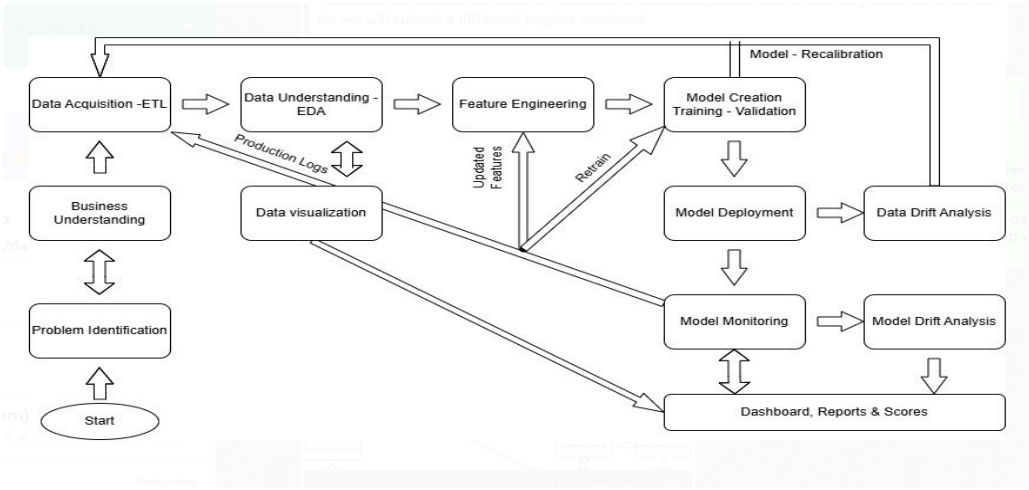


Figure 1. *Data Science Life Cycle*

Problem Identification:

Navigating urban environments to find available parking spaces has become an increasingly significant challenge in densely populated cities worldwide. This issue not only causes frustration for drivers but also leads to substantial economic, environmental, and societal costs.

In the United States, for instance, drivers spend an average of 17 hours annually searching for parking, resulting in a cumulative cost of approximately $73 billion per year due to wasted time, fuel, and increased emissions . Such "cruising for parking," contributes to up to 30% of traffic congestion in urban centers.

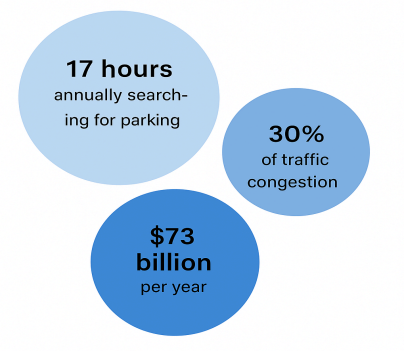


Figure 2. *Magnitude Of Problem*

The environmental implications are equally concerning. The additional vehicle emissions from prolonged idling and circling exacerbate air pollution, impacting air quality and public health.

Moreover, the economic burden extends beyond individual drivers. The demand for parking spaces influences urban planning decisions, often leading to increased construction costs and reduced land availability for other essential infrastructures. For example, in cities like New York, monthly parking costs can exceed $700, reflecting high value and scarcity of urban land.

Addressing these challenges necessitates intelligent parking systems to optimize parking space utilization, which in turn, reduces environmental impact, and enhances the overall efficiency of urban transportation networks.

Translating the Problem into a Data Science Challenge:

The above difficulties associated with urban parking management, can be effectively modeled as a data science problem in the domain of CV . Specifically, the **task of determining the occupancy status of parking spaces** lends itself to **image classification** and **object detection,** two core subfields in deep learning. Each frame captured by a camera can be interpreted as an image containing multiple subregions, each corresponding to a potential parking slot. **The central objective becomes a binary classification problem**: predicting

whether a given slot is occupied or vacant based on visual cues. We can use **supervised learning**, where deep learning architecture can be trained on **labeled image patches**.

In more dynamic and complex scenarios mere image classification may not suffice. In such cases, **object detection** methods like the **YOLO** family of algorithms are better suited, **which are beyond the scope of this project.**

In addition to classification and detection, certain deployments may benefit from **image segmentation** techniques. Here, each pixel in the image is assigned to a class enabling a more granular spatial understanding of the scene. **This is also beyond the scope of this project**.

Thus, what begins as an urban infrastructure problem translates into a multi-layered CV problem involving classification, detection, and segmentation, all of which are well addressed through deep learning approaches.

Data Acquisition

In this study, we used the publicly available PKLot dataset which has become a benchmark for CV research in this domain.It consists of 12,417 parking lot images (1280×720 pixels) captured using a low-cost HD-5000 webcam, installed on the rooftops of campus buildings to minimize vehicle occlusion. The images were recorded every five minutes over a span of more than 30 days, **covering diverse weather conditions—sunny, rainy, and overcast.** The images were sourced from three locations and captured from different floors, **ensuring variability in camera angle and elevation.**

Data Understanding

The PKLot dataset was constructed through a three-step process: **image acquisition**, **manual labeling**, and **segmentation**. An interactive annotation tool was used to mark the limits and labels of valid parking spaces in each image. Only well-marked parking spaces were included.

During segmentation, **skew correction** was applied to normalize the orientation of each cropped image, **ensuring consistency across samples**.

The resulting dataset provides:

- Diverse environmental conditions (lighting and weather variability),
- Different parking lot perspectives and structural characteristics,
- Visual challenges such as shadows, overexposed areas, and rainy conditions,
- Balanced class distribution with ~48.5% occupied and ~51.5% vacant slots overall.
- **No need to do object detection and annotation** of images manually or by YOLO **as each image is accompanied by an XML annotation file** containing bounding box coordinates for each individual parking space in the image and the occupancy status (vacant or occupied) of each parking slot. From these annotated images, over 695,000 individual parking space crops were generated and assigned binary labels to each: 0 (vacant) or 1 (occupied).

Such variability enhances the generalizability and robustness of models trained on this data.

Scope and Broad Direction Of project:

Scope: Scope is limited to do **image classification** only **presenting a comparative evaluation of four different models** and to make a Gradio app prototype.

Broad Direction: Create four different models (CNN, ViT, ResNet50-TF and ResNet50-Torch) following standard machine learning training and evaluation practices as below, choose the best model and deploy it. Next section describes a brief explanation of the algorithm for the ViT model that was eventually chosen for deployment. Similar explanation for CNN-from scratch is given in Appendix D as D1

Vision Transformer Algorithm:

1. Theoretical Foundation

The **Vision Transformer (ViT)** is a deep learning architecture introduced by Dosovitskiy et al. (2020) that applies the **transformer model architecture**, originally developed for natural language processing (NLP), to image classification tasks. Unlike CNNs that process local spatial neighborhoods using convolutions, ViTs treat images as a sequence of patches and process them using global **self-attention** mechanisms. ViT demonstrates that a pure transformer applied directly to sequences of image patches can work exceptionally well on image classification tasks.and it is not mandatory to use ResNet, YOLO etc. which depend on CNN.

2. **Core Architecture and Mathematical Description (Figure 3)**

(a) Image to Patch Embedding**:** The input $x \in R^{H \times W \times C}$ is split into non-overlapping patches of size P×P. Each patch is flattened and linearly projected to form a vector of fixed dimension D. Number of patches: $N = \frac{HW}{P^2}$

Each patch: $x_p^{(i)} \in R^{p^2 \cdot C} \rightarrow Linear\ Projection\ -> z_0^{(i)} \in R^D$

(b) Positional Embeddings:

Since transformers are permutation-invariant, learnable positional embeddings $E_{pos} \in R^{N \times D}$ are added: $z_0 = [x_{cls}; x_p^{(1)}E; \ldots\ldots\ldots\ldots ; x_p^{(N)}E] + E_{pos}$ where x_{cls} is a special learnable [CLS] token used to aggregate image-level representation

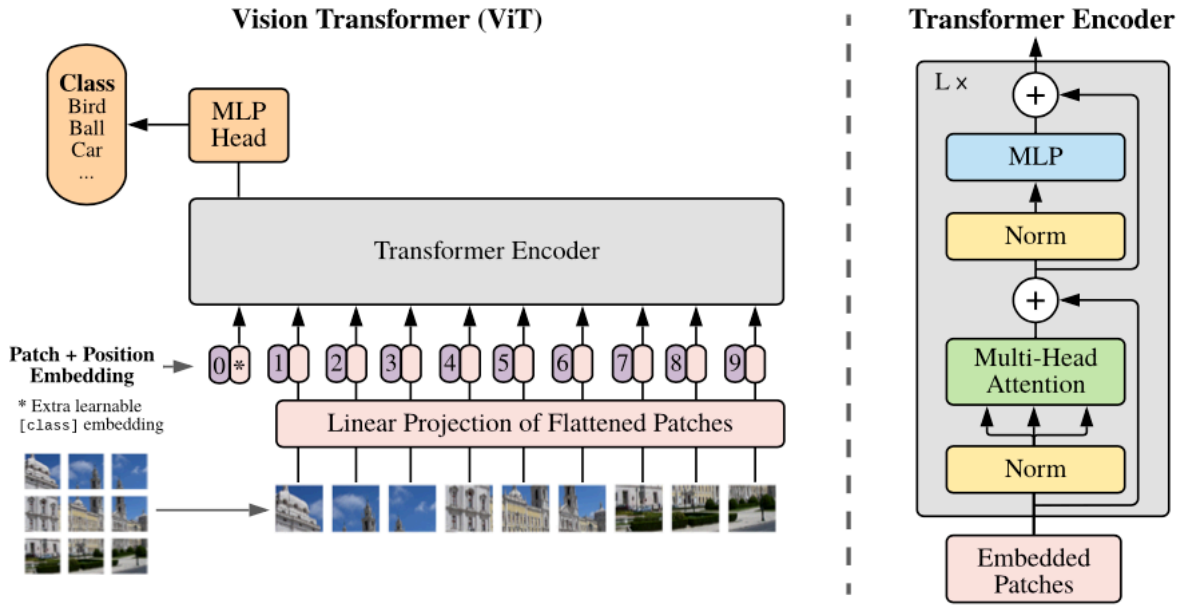


Figure 3. *Vision Transformer Architecture*

(c) Transformer Encoder:

Each encoder block contains:

- Multi-head self-attention (MSA):
- Feed-forward network (MLP):
- Layer normalization and residual connections:

$$z'_l = MSA(LN(z_{(l-1)}) + z_{(l-1)}$$

$$z_l = MLP(LN(z'_1) + z_{(l-1)}$$

Here l=1…L, where L is the number of transformer layers.

(d) **Classification Head:** The final representation of the [CLS] token is passed through a dense classifier head: $\hat{y} = \sigma(W_{ZL}^{|CLS|} + b)$ Where σ is the sigmoid function for binary classification.

Feature Engineering:

Continuing the methodology (Figure 1) is feature engineering. As we have to describe this for four models, for brevity, only feature engineering for ViT model is described below. However the process for the other three models is placed in Appendix in table A1.

Steps for ViT Model (Vision Transformer - TensorFlow)

1. **Image Resizing :** Images are resized using image_size=(IMG_SIZE, IMG_SIZE) in the image_dataset_from_directory() function.
2. **Normalization :** Images are manually normalized using a map function: each pixel is scaled by dividing by 255.0 (i.e., x/255.0).
3. **Data Augmentation:** Applied via layers such as RandomFlip("horizontal") and RandomRotation(0.1) to improve generalization.
4. **Label Encoding:** Labels are encoded using the label_mode='int' option when loading the dataset—typically assigning 0 or 1 based on folder names.
5. **Dropout:** Dropout is applied in the dense layers before the final sigmoid output layer to prevent overfitting.
6. **Early Stopping:** Implemented with a callback that monitors validation loss, often combined with ModelCheckpoint to restore the best weights.
7. **Input Format:** Data is loaded as **TensorFlow datasets** using image_dataset_from_directory, compatible with model training workflows.

Model Creation:

The next stage in the ML life cycle is model creation. Table 2 describes the summary of architecture of the ViT model. Table A2 in Appendix A shows architecture for other models.

Table 2

Model Architecture Summary for Vision Transformer model.

Sr No.	Model	Model Architecture
1	ViT	Base model: vit_keras.vit_b16 pretrained on ImageNet • trainable=True (fine-tuned) • Output from base → Flatten → Dense(128) → Dropout(0.1) → Dense(1, activation='sigmoid')

Training Process Summary (All Models)

The training process for each model can be understood by how training and validation accuracy and training and validation loss change with epoch. Figure-3 shows the plots for the ViT model However, similar plots for other three models are also given in Appendix C (Figure C1,C2,C3) and highlights of the learning process for all models are given after Figure-3.

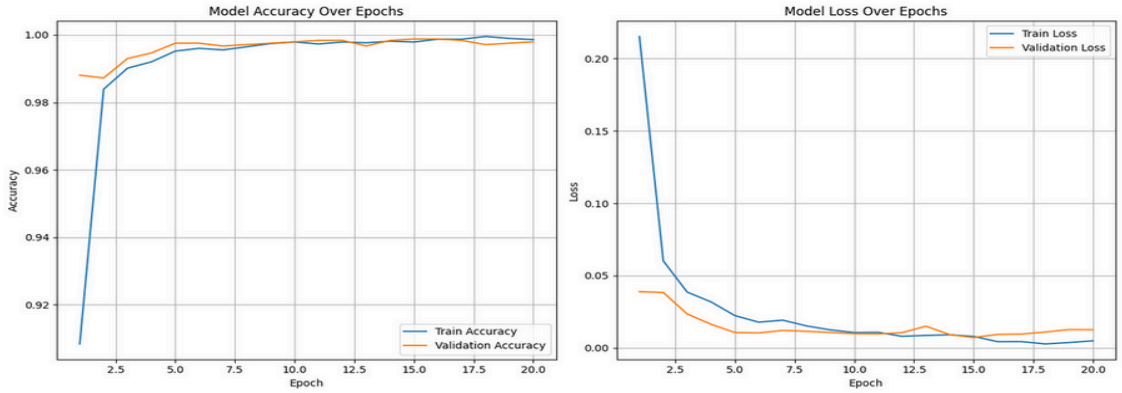


Figure 4. *Training Plot For ViT*

Highlights of Model Training Across Experiments

- Models improved steadily, with no signs of overfitting—validation accuracy stayed high.
- Data augmentation, dropout, and learning rate scheduling enhanced generalization.
- Early stopping was used to restore the best model weights.

- **Vision Transformer** had the best performance, peaking at 99.88% accuracy with a validation loss of 0.0072.
- **CNN from scratch** peaked at 88% accuracy by Epoch 5.
- **ResNet-based models** (TF and PyTorch) reached ~92–93% accuracy, stabilizing by later epochs.

Use Of Precision-Recall (P-R) curve To find optimal classification threshold:

In any classification, the classification threshold plays a crucial role in model-performance and the default threshold of 0.5 is often not the best for real world applications. For example,

- **If our app shows drivers where to park** → ***recall*** *is more important (so they don't go to an already-occupied spot).*
- **If our app enforces rules (e.g., penalties for occupying unauthorized spots)** → ***precision*** *is more important (avoid false accusations).*

Normally **the threshold that Gives best F-1 score** is selected, which can be found using P-R curve.The P-R curves for validation set for the best model is shown in Figure 4. For the rest of the models they are in Appendix C (plots C4,C5, C6). However, brief summary insights for all models is given after Figure-4

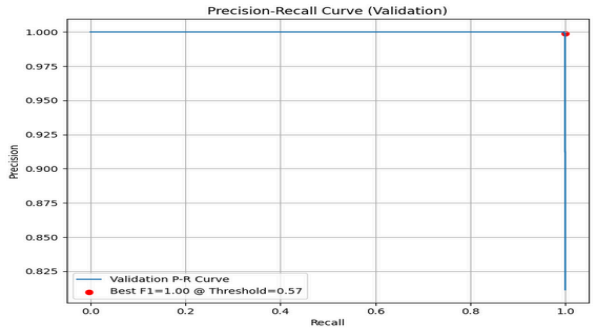


Figure 5. *P-R curve For ViT Model (valid set)*

Summary Insight From P-R curves:

- ViT and ResNet50-Torch deliver perfect P-R performance, but ViT is better calibrated.

- ResNet50-TF is slightly behind but robust and well-balanced.
- CNN performs commendably, though with less confidence in predictions

Comparative Model Evaluation Results:

Comparative evaluation results for important metrics are shown in Figure-5,6,7,8,9 below , showing that ViT and Resnet50-Torch models both have comparable performance but the threshold of 0.03 for Resnet-Torch and its unstable training curve tilted the balance in favour of ViT model as the best model

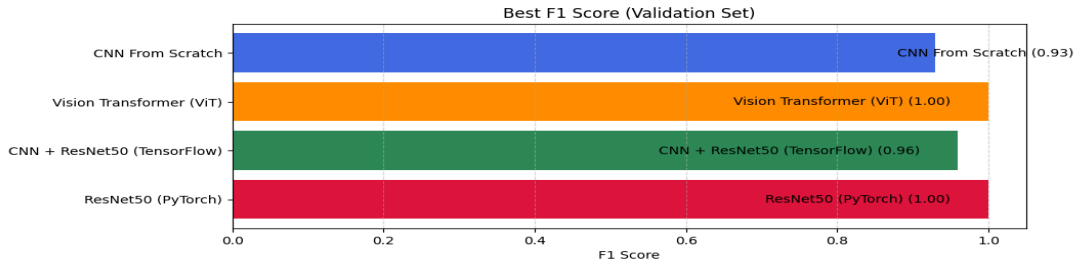


Figure 6 *Comparison Of Best F-1 Scores for all models*

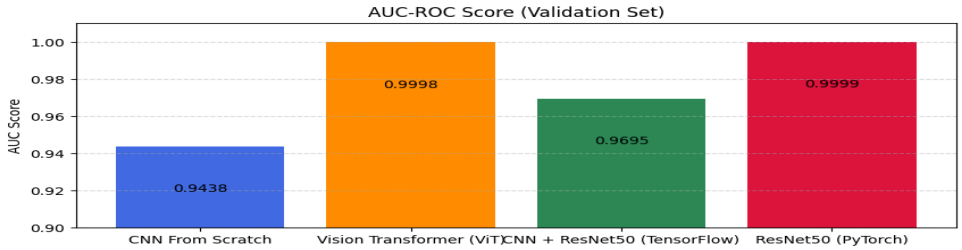


Figure 7 *Comparison Of AUC-ROC Score For All Models*

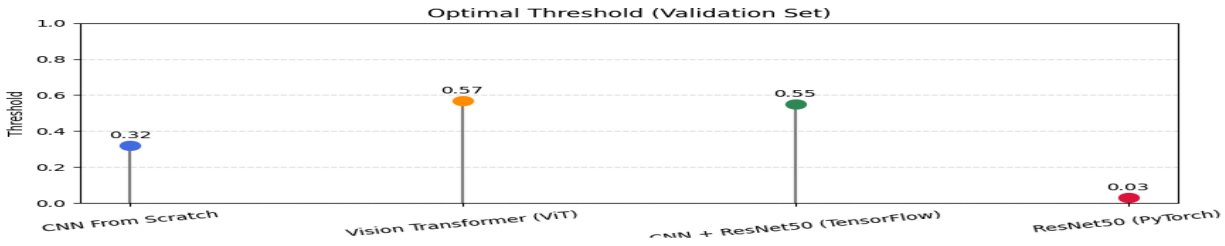


Figure 8. *Comparison Of Optimal Thresholds For All Models (Validation Set)*

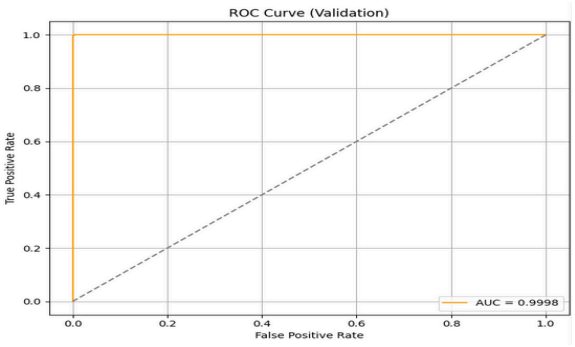


Figure 9 *ROC plots for ViT model (See Figures C7,C8,C9 in Appendix C for other models)*

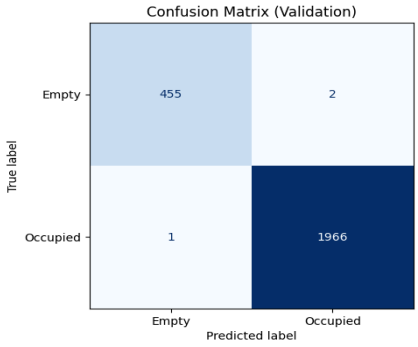


Figure 10 *Confusion matrix for ViT model*

Table 3

Results From Classification Report On Validation Set By Vision Transformer Models

Model Name	Precision (Empty)	Recall (Empty)	F1 (Empty)	Support (Empty)	Threshold	Best F1	AUC-ROC
Vision Transformer (ViT)	1.00	1.00	1.00	457	0.57	1.00	0.9998

Conclusion:

The results obtained by all four models are good. Especially there is a close contest between three models except CNN from scratch. However, the Vision Transformer based model was selected for deployment as its training curve shows that it would have more stability and generalizability and it gives perfect metrics. The ResNet50 (PyTorch) model also gives excellent results but its optimal threshold of 0.03 may not be appropriate for all production set-ups.

Deployment:

We chose the Vision Transformer based model as the best model and made a gradio app showing working of the model when hosted on a local server. Images of its WebUI are placed in Appendix C as C13.

Model Monitoring, Model Drift Analysis And Data drift Analysis:

Machine learning models' performance starts deteriorating after deployment due to 'data-drift' or 'model-drift'. So, continuous model monitoring and triggering a retraining is required when such a drift is observed. As this advanced discussion is beyond the scope of the project we will just list some tools for this like 'MLFLOW', 'APACHE AIRFLOW' and 'EVIDENTLY AI'. Using this a fully automated deployment , that automatically triggers re-training and restores performance.

Limitations:

The model performance for all models looks excellent because of the high quality dataset. However in practice, designing a real time solution on unannotated images presents new challenges. Hence, we find it appropriate to list a few limitations of this study. Their detailed discussion is done in Table A6 in Appendix A.

Figure 11. *Limitations Of Current Project*

Future Scope: Detailed discussion is placed in Appendix A as table A5

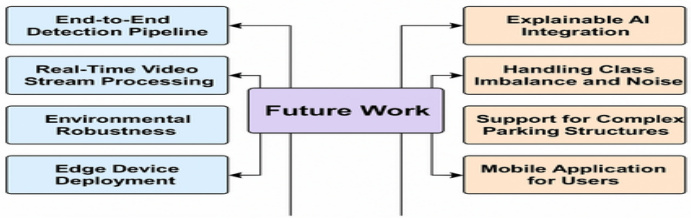


Figure 12. *Future Scope*

References

Almeida, P., Oliveira, L. S., Silva Jr, E., Britto Jr, A. S., & Koerich, A. (2015). PKLot – A robust dataset for parking lot classification. *Expert Systems with Applications, 42*(11), 4937–4949. https://doi.org/10.1016/j.eswa.2015.02.009

Noor, M., & Shrivastava, A. (2021). Automatic parking slot occupancy detection using Laplacian operator and morphological kernel dilation. In *10th IEEE International Conference on Communication Systems and Network Technologies* (pp. 825). IEEE. https://doi.org/10.1109/CSNT.2021.145

Porvatov, V., Tishin, V., Kuznetsov, A., Semenova, N., Martynova, A., Kuznetsov, M., & Kuznetsova, K. (2023). Revising deep learning methods in parking lot occupancy detection. *arXiv preprint* arXiv:2306.04288. https://doi.org/10.48550/arXiv.2306.04288

Sahoo, B. B. P., Shahjad, & Tanwar, P. S. (2021). Real-time smart parking: Challenges and solutions using machine learning and IoT. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology, 7*(2), 451–458. https://doi.org/10.32628/CSEIT217295

Števaňák, R., Matejov, A., Jariabka, O., Šuppa, M., Nagy, M., & Farkaš, I. (2017). PKSpace: An open-source solution for parking space occupancy detection. In *Proceedings of the 21st Central European Seminar on Computer Graphics* (non-peer-reviewed).

Wei, L., Cao, L., Yan, L., Li, C., Feng, X., & Zhao, P. (2020). Vacant parking slot detection in the around view image based on deep learning. *Sensors, 20*(7), 2138. https://doi.org/10.3390/s20072138

Zhang, L., Huang, J., Li, X., & Xiong, L. (2018). Vision-based parking-slot detection: A DCNN-based approach and a large-scale benchmark dataset. *IEEE Transactions on Image Processing, 27*(11), 5350–5360. https://doi.org/10.1109/TIP.2018.2857407

Unpublished or Institutional Papers (n.d.):

Cazamias, J., & Marek, M. (n.d.). *Parking space classification using convolutional neural networks*. Stanford University. jaycaz@stanford.edu & mmarek@cs.stanford.edu

Kim, S., Kim, J., & Lee, K. (n.d.). *A vision-based parking lot management system using machine learning*.

Kumar, A., & Agrawal, K. (n.d.). *Parking space finder using image processing and machine learning*. Department of Computer Science & Engineering, Integral University Lucknow.

Levkivskyi, S., & Marchuk, V. (n.d.). *Available parking places recognition system*. Semantic Scholar. https://www.semanticscholar.org/paper/Available-parking-places-recognition-system-Levkivskyi-Marchuk/f0a5be9497597c093e85b4e266a3f862f26862cb

Li, Y., Zhang, J., Salahuddin, M., Miah, M. A. R., Vega-Rodriguez, M. G., Jimenez-Ramirez, A., & Alvarez-Icaza, L. (n.d.). *A comprehensive study of real-time vacant parking space detection: Towards the need of a*.

Park, S. H., Kim, C. S., & Ko, Y. B. (n.d.). *Parking lot occupancy detection using machine learning and image processing*.

Tschentscher, M., & Neuhausen, M. (n.d.). *Video-based parking space detection*. Institute for Neural Computation, Ruhr-Universität Bochum. marc-philipp.tschentscher@ini.rub.de

Web Sources:

INRIX. (n.d.). *Parking pain in the U.S.* https://inrix.com/press-releases/parking-pain-us/

MJE. (2022, June 20). *The economics of parking*. University of Michigan Journal of Economics. https://sites.lsa.umich.edu/mje/2022/06/20/the-economics-of-parking/

Wall Street Journal. (n.d.). *Donald Shoup, UCLA economist, obituary*. https://www.wsj.com/us-news/donald-shoup-ucla-economist-obituary-5c555ccc

Wired. (n.d.). *Mexico City is killing the parking spot*. https://www.wired.com/story/mexico-city-killing-parking/

Viso.ai. (2023, November 25). *Vision Transformers (ViT) in Image Recognition: Full Guide*. https://viso.ai/deep-learning/vision-transformer-vit/

Appendix A

Table A1

Feature Engineering For Other Models

Step	Technique	CNN (from scratch)	ResNet50 + Custom CNN (TF)	ResNet50 (PyTorch)
1	Image Resizing	target_size=(224, 224) via ImageDataGenerator	target_size=(224, 224) in ImageDataGenerator	transforms.Resize((224, 224))
2	Normalization / Rescaling	rescale=1./255	rescale=1./255	transforms.ToTensor() + Normalize(mean, std)
3	Data Augmentation	rotation_range=90, horizontal_flip, vertical_flip	Same as CNN: 90° rotation + flipping	Not applied
4	Label Encoding	class_mode='binary' (auto label from folder names)	class_mode='binary'	ImageFolder() auto-encodes folder labels
5	Dropout	Used in final layers to reduce overfitting	Dropout in CNN block before dense layers	Dropout not explicitly mentioned
6	Early Stopping	Yes (monitors val_loss)	Yes	Not specified
7	Input Format	Keras image batches	Keras flow_from_directory()	PyTorch Dataset + DataLoader

Table A2

Model Architecture Summary for other models

S No.	Model	Model Architecture
1	CNN from Scratch	Custom Sequential model with: • 3× Conv2D + MaxPooling2D layers • Flatten → Dense(128) → Dropout(0.5) • Output layer: Dense(1, activation='sigmoid') for binary classification
2	ResNet50 + Custom CNN (TF)	Base: ResNet50(include_top=False, weights='imagenet') • Base frozen (no retraining) • Followed by Flatten → Dense(128) → Dropout(0.5) → Dense(1, activation='sigmoid')
3	ResNet50 (PyTorch)	torchvision.models.resnet50(pretrained=True) • fc layer replaced with: nn.Sequential(nn.Linear(2048, 128), nn.ReLU(), nn.Dropout(0.5), nn.Linear(128, 1)) • Output activated with sigmoid for binary output

Table A3

Training Process Summary (All Other Models)- Tabular Representation

Model	Best Validation Accuracy	Epoch of Peak	Validation Loss Trend	Overfitting Observed?	Key Regularization Used
CNN from Scratch	88.00%	Epoch 5	Decreased till Epoch 5, slight rise thereafter	No	Data Augmentation + Dropout
ResNet50 + Custom CNN (TF)	93.00%	Epoch 20	Gradual drop; stable near 0.17 at end	No	ReduceLROnPlateau + Dropout
ResNet50 (PyTorch)	92.88%	Epoch 14	Smooth decline, no spike	No	Early stopping, LR scheduler

Table A4

Summary Table From Precision-Recall Curves For All other Models(Validation-Set)

Model Name	Best F1 Value	Best Threshold	AUC-ROC Score
CNN From Scratch	0.93	0.32	0.9438
CNN + ResNet50 (TensorFlow)	0.96	0.55	0.9695

ResNet50 (PyTorch)	1.00	0.03	0.9999

Table A5

Results From Classification Report On Validation Set By All other Models

Model Name	Precision (Empty)	Recall (Empty)	F1 (Empty)	Support (Empty)	Threshold	Best F1	AUC-ROC
CNN From Scratch	0.65	0.93	0.77	457	0.32	0.93	0.9438
CNN + ResNet50 (TensorFlow)	0.77	0.93	0.84	457	0.55	0.96	0.9687
ResNet50 (PyTorch)	1.00	1.00	1.00	457	0.03	1.0000	0.9999

Table A6

Future Scope

Focus Area	**Description**
End-to-End Detection Pipeline	Integrate object detection models (e.g., YOLO, SSD) to detect and localize parking slots from full-frame images, removing the need for pre-segmented inputs.
Real-Time Video Stream Processing	Extend the system to handle continuous surveillance video streams, enabling real-time slot updates with temporal consistency.
Environmental Robustness	Use datasets with more diverse conditions (e.g., night, snow, fog) and apply techniques like domain adaptation or GAN-based augmentation to improve generalization.
Edge Device Deployment	Optimize models for deployment on devices like Raspberry Pi or Jetson using pruning, quantization, or lightweight architectures like MobileNet or TinyViT.
Explainable AI Integration	Apply interpretability tools such as Grad-CAM (for CNNs) and attention map visualizations (for ViTs) to enhance transparency and trust.
Handling Class Imbalance & Noise	Use advanced methods like focal loss, label smoothing, or semi-supervised learning to mitigate class imbalance and noisy labels.
Support for Complex Structures	Extend the solution to work with multi-level or underground parking facilities that have varying layouts and lighting conditions.

Mobile Application for Users	Develop a user-facing mobile app with QR-based access to enable live viewing, reservation, and navigation to available slots.
Smart City Integration	Integrate with urban mobility platforms and smart city dashboards for real-time data sharing, predictive analytics, and automated enforcement.

Table A7

Limitations

S No.	Category	Limitation	Explanation
1	Dataset Generalization	PKLot is captured from fixed locations	All images are from two university campuses in Brazil (UFPR & PUCPR), captured under specific angles and camera heights. The model may not generalize well to parking lots with drastically different layouts, camera angles, or environmental conditions.
2	Static Images Only	No real Time Video Processing	The current pipeline uses static images. In real-world scenarios, a real-time system must process video feeds and detect status changes dynamically.
3	Pre-segmented Input	Assumes pre-cropped parking slots	Models work on cropped images of individual slots provided by PKLot. In deployment, the system must first detect parking slot regions from full-frame lot images (requiring object detection or segmentation).
4	No End-to-End Detection	No localization of slots in full images	The project does not implement or test the object detection stage (e.g., YOLO), which would be essential for deployment in new lots where slot boundaries are unknown.
5	Binary Classification Only	Occupied/Vacant only — no confidence or intermediate states	Does not account for ambiguous or partially blocked views, motorbikes, improperly parked cars, or occluded spaces.
6	Environmental Variability	Weather and lighting variability are limited	While PKLot includes sun, rain, and cloudy conditions, it lacks night-time scenes, snow, fog, motion blur, etc., which may appear in real-world deployments.
7	Hardware Constraints Not Modeled	No latency or inference time analysis	The models may be heavy (especially ViT or ResNet) and may not run efficiently on edge devices (e.g., Raspberry Pi, Jetson Nano)

			without quantization or pruning.
8	Limited Explainability	Black-box nature of ViT/ResNet	No SHAP/LIME/GradCAM explainability techniques are used to interpret the model's decisions. For critical applications, this might limit trustworthiness.
9	Dataset Biases	Possibility of annotation errors or label imbalance	Some slots may be consistently empty or consistently occupied, biasing the model. There's also the risk of human error in manual annotations.
10	Scalability to Multi-level Parking	Not evaluated on complex parking structures	The model is trained on open ground-level parking lots. Multi-level or underground structures with varied lighting and layout are not considered.

Appendix B
Repository Information

The full source code used in this project is available at the following two repositories:

(a) Bandaru, A., Bhatija, R. & Desai, T (2025). Project-AAI-501 [Source code]. GitHub. https://github.com/aimldstejas/CV_PROJECT_PARKING_LOT_DETECTION

(b) Bandaru, A., Bhatija, R. & Desai, T (2025). Project-AAI-501 [Source code]. GitHub. https://github.com/Anusha-twelve/Project-AAI-501

This repository contains all scripts for data preprocessing, analysis, and visualization. Users can access the latest updates and documentation in the repository's README file.

Appendix C

Supplementary important diagrams

Figure C1

Training Plot For CNN Model

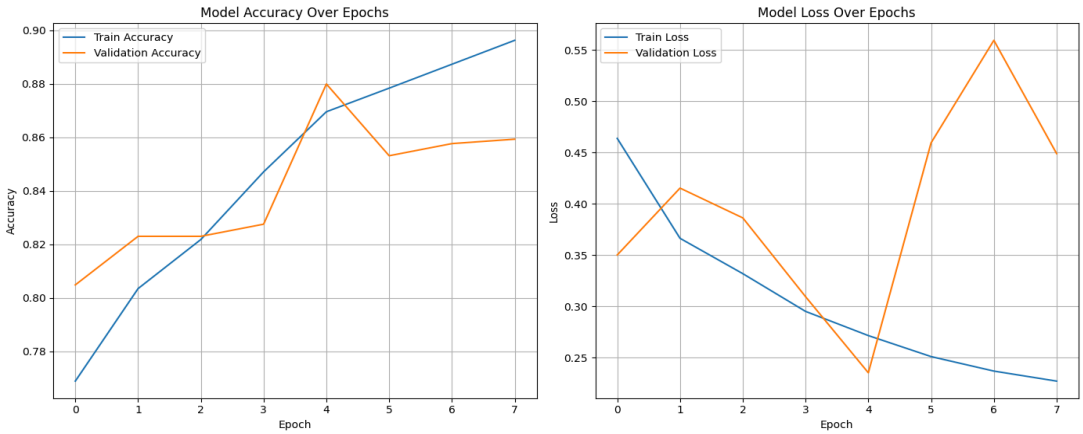


Figure C2

Training Plot For ResNet50-TF

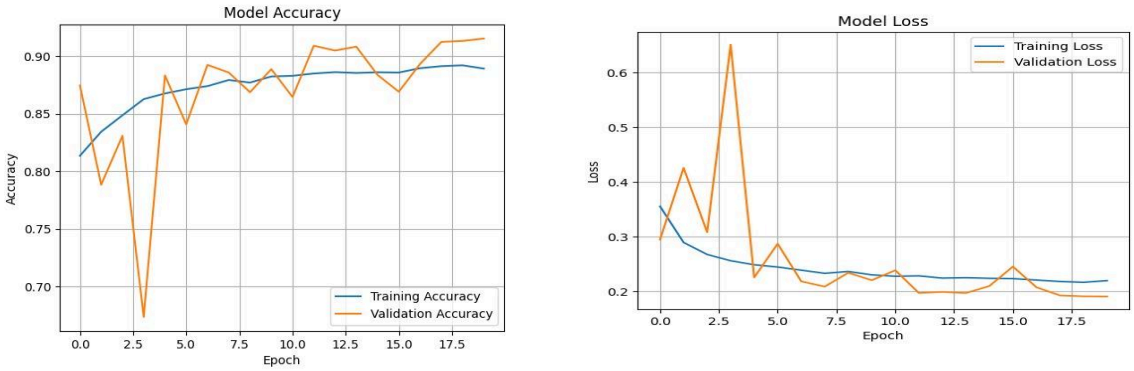


Figure C3

Training Plot For ResNet50-Torch Model

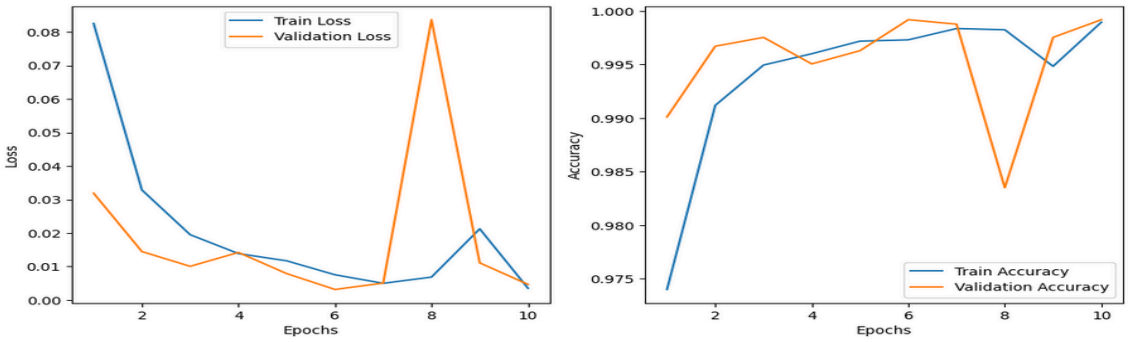


Figure C4

P-R curve For CNN (valid set)

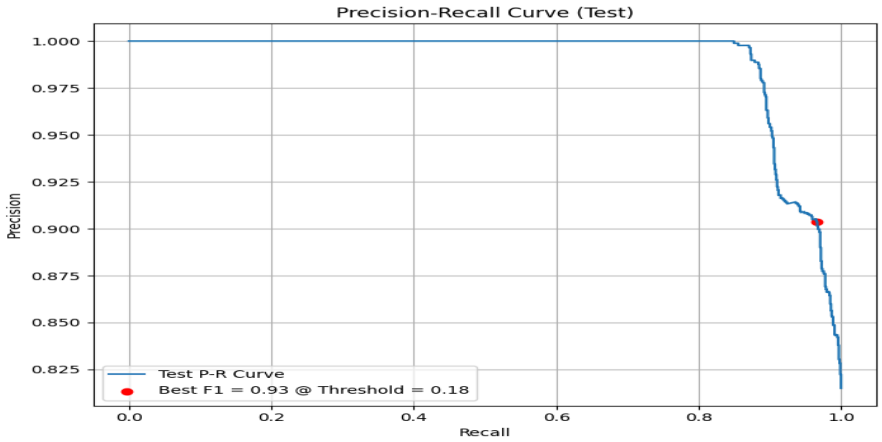


Figure C5

P-R curve For Resnet50-TF (valid set)

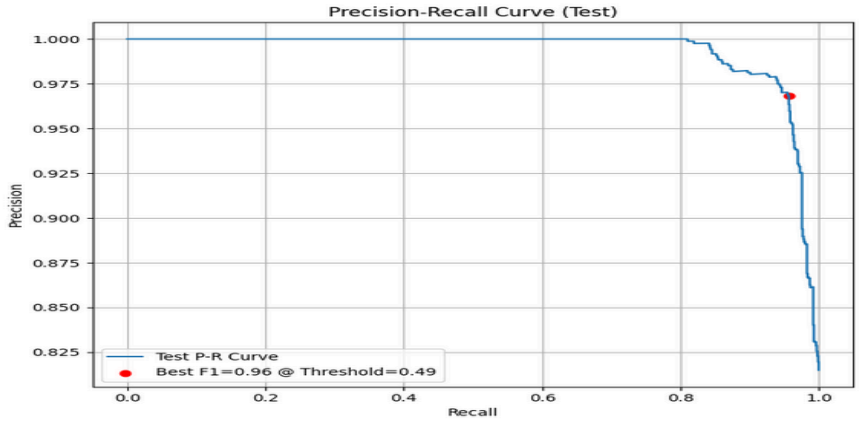


Figure C6

P -R curve For Resnet50 -Torch (valid set)

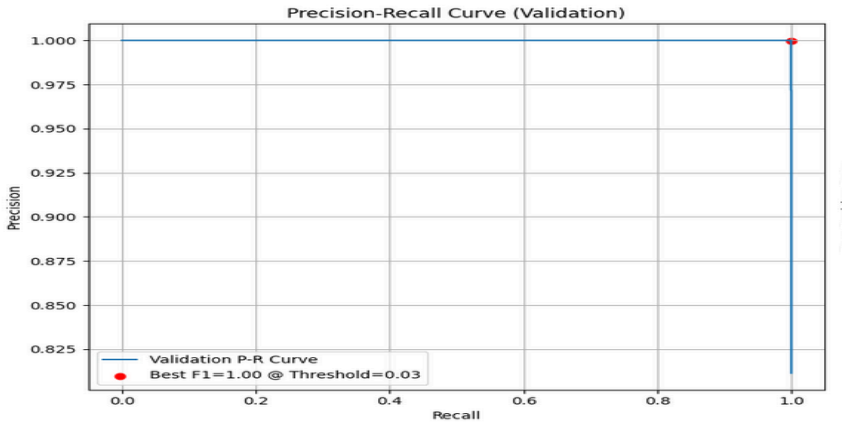


Figure C7

ROC Plot For CNN From Scratch

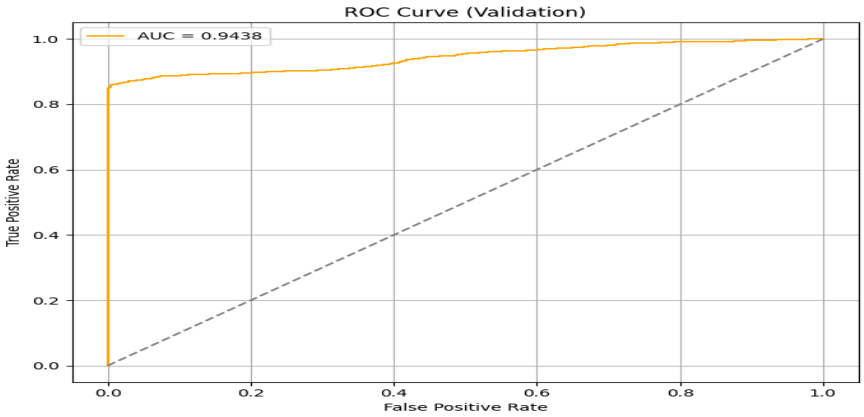


Figure C8

ROC Plot For Resnet50 TensorFlow Model

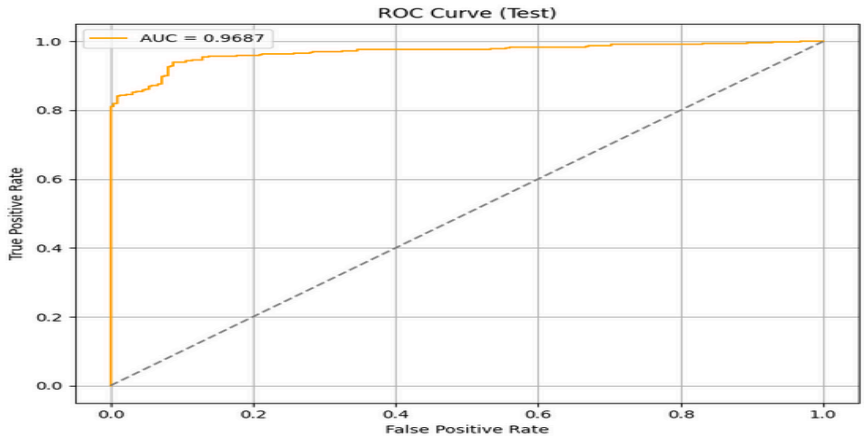


Figure C9

ROC Plot For ResNet50 Pytorch Model

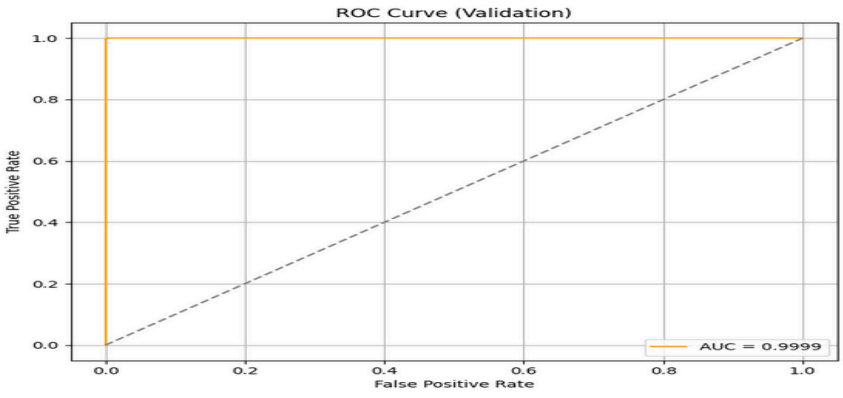


Figure C10

Confusion Matrix For CNN From Scratch

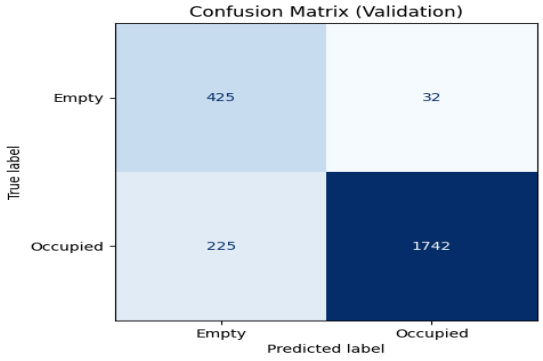


Figure C11

Confusion Matrix For Resnet50 TensorFlow Model

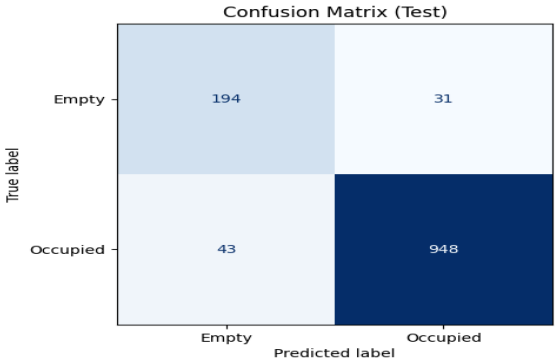


Figure C12

Confusion Matrix For ResNet50 Pytorch Model

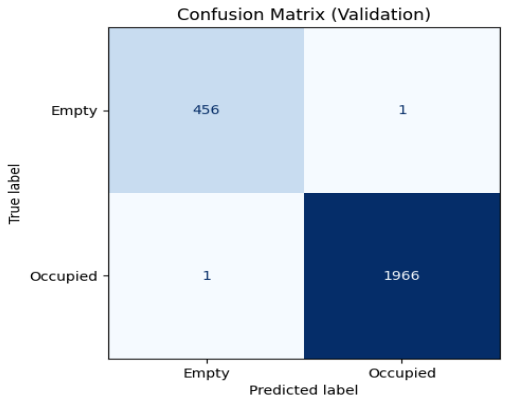
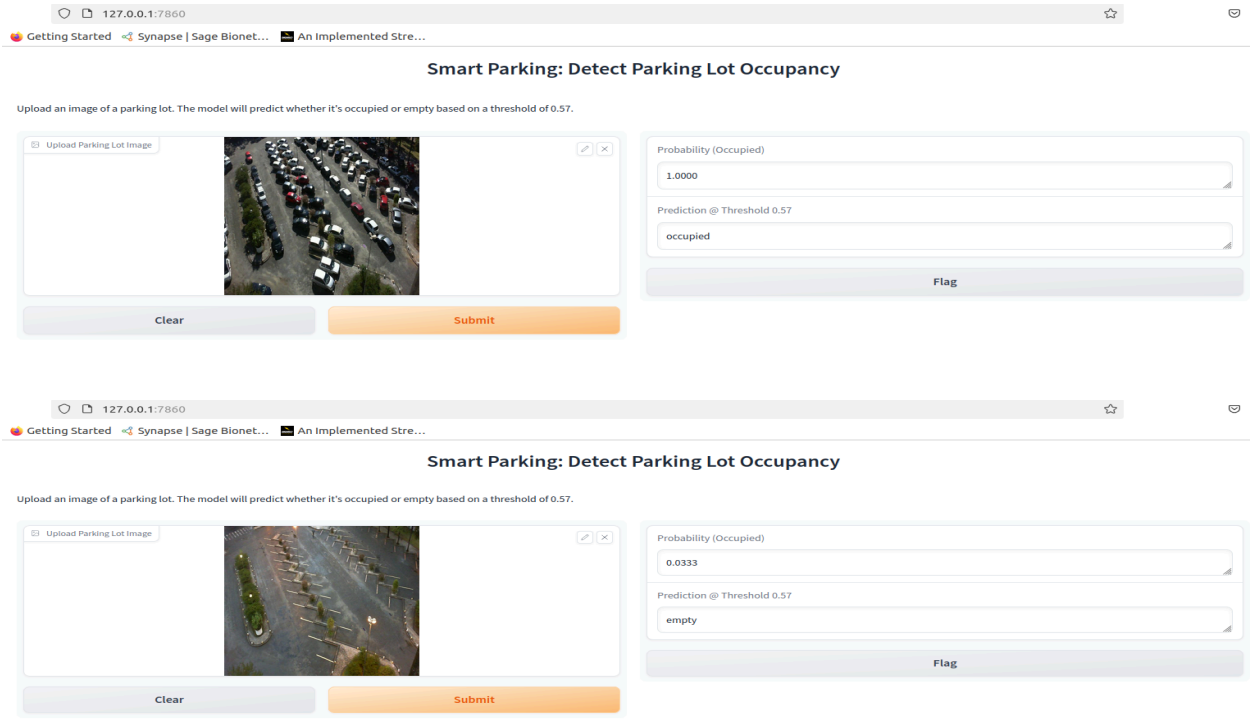


Figure C13

Web based App UI



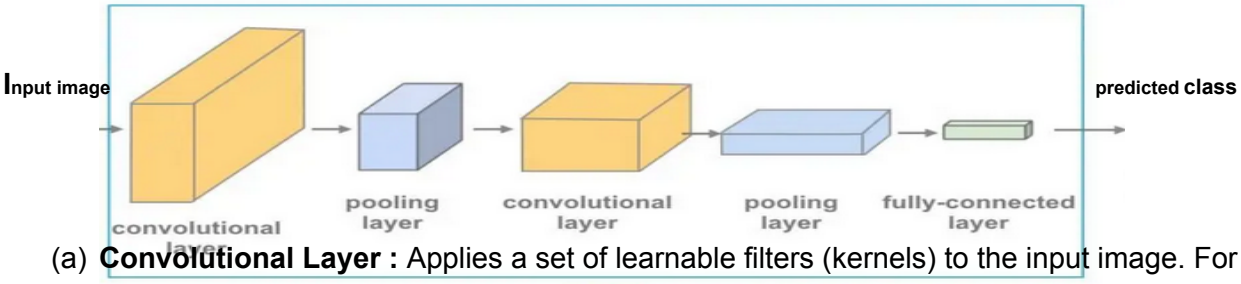
Appendix D

D1 *Architecture Of CNN from scratch*

1. Theoretical Foundation

A **Convolutional Neural Network (CNN)** is a deep learning architecture designed specifically for processing data with a grid-like topology, such as images. Unlike fully connected networks, CNNs leverage spatial locality by enforcing a local connectivity pattern between neurons of adjacent layers.

Key Components and Mathematical Foundations:



(a) **Convolutional Layer :** Applies a set of learnable filters (kernels) to the input image. For a 2D input X and filter K, convolution output is:

$$(X \cdot K)(i,j) = \sum_m \sum_n X(i+m, j+n) \cdot K(m,n)$$

This operation extracts spatial features such as edges, corners, or textures.

(b) **ReLU Activation:** Applies non-linearity to the feature maps: $f(x) = max(0,x)$

(c) **Pooling Layer:** Reduces the spatial dimensions, retains the most prominent features:

$$MaxPool(X) = max\, X(i,j)\ where\ i,j \in R$$

(d) **Flatten Layer:** Converts 2D feature maps into a 1D vector for feeding into dense laye

(e) **Fully Connected (Dense) Layer:** Performs high-level reasoning using learned features.

Output is typically: $y = \sigma(Wx + b)$ where σ is sigmoid activation

(f) **Dropout Layer:** Disables a fraction p of neurons during training. Prevents overfitting.

(g) Loss Function (Binary Cross Entropy): For a true label y∈{0,1} and predicted probability $\widehat{y}$

$$L = -[y log(\widehat{y}) + (1-y) log(1-\widehat{y})$$